Line Bot 星座運勢指南：微服務架構與雲端部署實踐

結合 Python、Line Messaging API 與 Render 雲端平台之開發成果

報告人：鍾名宣



Made with GAMMA

從本機腳本到 24 小時可用的服務

01

核心目標：

提供使用者 24 小時不間斷的即時星座運勢查詢服務。

02

痛點分析：

傳統本機執行（如 ngrok）無法長時間運作且容易中斷。

03

解決方案：

採用 Git 版本控制整合 GitHub，並將服務部署至 Render 免費雲端平台，實現永續服務。

Made with GAMMA

四層架構串起 LINE 訊息與運勢回覆

1. **使用者層**：LINE App 用戶發送訊息。
2. **訊息傳輸層**：LINE Messaging API（Webhook 機制）。
3. **雲端服務層**：Render Web Service 託管 Flask 後端。
4. **核心邏輯層**：Python 智慧解析星座關鍵字並回傳客製化運勢卡片。



Made with GAMMA

技術棧各司其職，支撐完整部署流程

後端開發：

Python 3、Flask 輕量化 Web 框架。

API 整合：

line-bot-sdk（LINE Messaging API）。

版本控制：

Git、GitHub 專案庫管理。

雲端託管：

Render 伺服器，使用 Gunicorn WSGI HTTP Server。

Made with GAMMA

從 Webhook 到個人化運勢卡片

1. **訊息監控**：設定 `/callback` 路由接收 LINE Webhook 請求。
2. **身份驗證**：運用 Channel Secret 與 Access Token 確保訊息安全性。
3. **關鍵字匹配**：自動辨識十二星座，如：天蠍座、雙魚座等。
4. **動態生成**：隨機抽取幸運顏色、幸運數字與每日建議，並打包為結構化回覆訊息。

```python
@app.route("/callback", methods=["POST"])
def callback():
    signature = request.headers.get("X-Line-Signature")
    body = request.get_data(as_text=True)

    try:
        handler.handle(body, signature)
    except InvalidSignatureError:
        abort(400)

    return "OK"


@handler.add(MessageEvent, message=TextMessage)
def handle_message(event):
    user_text = event.message.text.strip()
```

Made with GAMMA

部署排除障礙，讓服務穩定上線

挑戰

Git 提交時將巨大的 `.cache` 暫存檔寫入，導致推送到 GitHub 卡住。

解決方案

配置 `.gitignore` 忽略非必要檔，並以 `git rm -r --cached .` 清理暫存，大幅提升推送速度。

效能調整

了解 Render 免費實例（Free Instance）的 Spin Down 自動休眠機制，並優化首次喚醒回應時間。

Made with GAMMA

LINE Webhook 驗證成功，機器人正式上線

成果：

通過 LINE Webhook 驗證（Verify Success），機器人穩定上線。

跨平台使用：

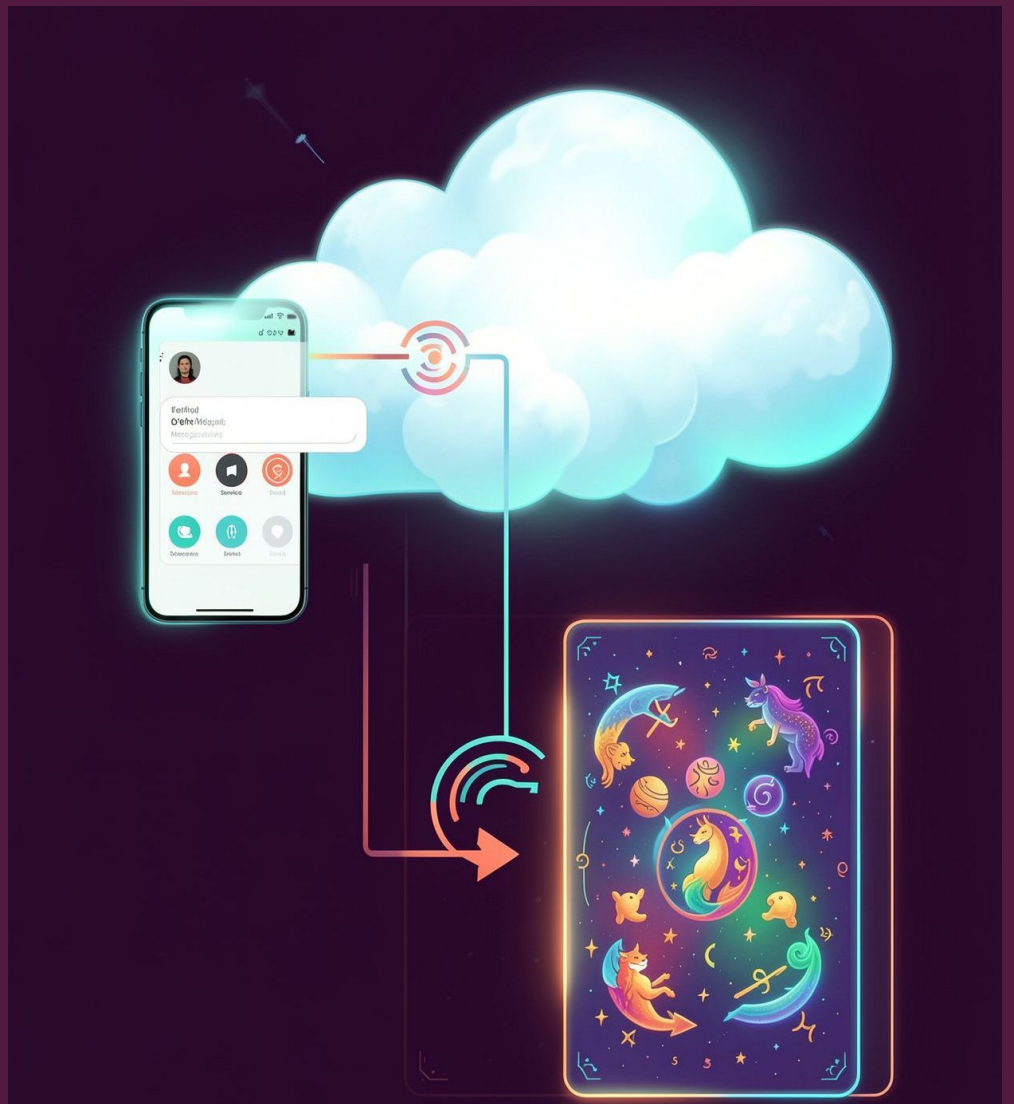
使用者只要開啟 LINE App 即可查詢。

免關機服務：

部署至雲端後，不需維持個人電腦開機。

自動回覆：

輸入星座關鍵字後，秒級收到星座運勢。



Made with GAMMA

完成從 API 開發到雲端部署的完整實踐

專案總結

成功整合 API 開發、版本控制與雲端部署完整流程。

未來擴充 1

串接真實氣象或運勢 API，提供即時運勢數據。

未來擴充 2

新增 Flex Message 視覺化圖卡，提升使用者體驗。

未來擴充 3

導入資料庫，如 PostgreSQL，儲存使用者偏好設定。

謝謝聆聽

Made with GAMMA